

Тестовое задание

Python Backend Developer

Ориентировочное время выполнения: 4–6 часов

Контекст задачи

Нужно реализовать небольшой backend-сервис для записи на встречи. Пользователь создаёт бронь через REST API, система ставит её в очередь, воркер асинхронно подтверждает запись и логирует моск-уведомление. Фронтенд не нужен — только API, воркер и тесты.

Требования к реализации

1. REST API

- POST /bookings — создать бронь (name, datetime, service_type)
- GET /bookings/{id} — статус брони (pending / confirmed / failed)
- GET /bookings — список броней с фильтром по статусу и пагинацией
- DELETE /bookings/{id} — отмена брони (только в статусе pending)

2. Фоновая обработка

- Celery-воркер получает задачу сразу после создания брони
- С вероятностью ~15% задача завершается ошибкой (имитация сбоя внешнего сервиса) — статус меняется на failed
- При успехе — статус confirmed, логируется моск-отправка уведомления
- Задача идемпотентна: повторный запуск с тем же booking_id не создаёт дубль и не ломает статус

3. Хранилище и инфраструктура

- PostgreSQL — миграции через Django ORM или Alembic
- Redis в роли broker и result backend для Celery
- docker-compose.yml поднимает весь стек одной командой: docker-compose up

- Переменные окружения через .env (шаблон .env.example обязателен)

4. Тесты

- pytest — покрытие API-эндпоинтов (happy path + граничные кейсы)
- Минимум один тест на логику воркера с моком внешнего вызова
- Тесты запускаются без поднятого Docker: pytest из корня

Будет плюсом

- Retry с экспоненциальным backoff для упавших задач
- Structured logging в формате JSON
- Простой rate limiting на POST /bookings
- Async FastAPI + TaskIQ вместо Celery — как альтернатива
- Makefile с командами dev / test / lint

Как сдать задание

Опубликуйте публичный репозиторий на GitHub и пришлите ссылку. В репозитории должен быть README со следующим:

- Как запустить сервис (docker-compose up + любые нужные шаги)
- Как запустить тесты
- Какие технические решения вы приняли и почему (выбор фреймворка, структура задачи, подход к идемпотентности)

Совет

README читается первым. Это первое впечатление о вас как о разработчике — уделите ему время.

Критерии оценки

Область	Вес	На что смотрим
---------	-----	----------------

Работоспособность	30 / 100	Сервис запускается, API отвечает, воркер обрабатывает задачи
Код и архитектура	25 / 100	Структура проекта, читаемость, отсутствие лишней сложности
Очереди и async	20 / 100	Правильная интеграция Celery/Redis, идемпотентность задачи
Тесты	15 / 100	Осмысленные кейсы, читаемые assertions, тесты действительно проходят
README и документация	10 / 100	Понятно, какие решения приняты и почему

Стоп-сигналы — отказ

Если хотя бы один из пунктов ниже верен — мы не приглашаем на следующий этап:

- docker-compose up не поднимает стек без ручных правок
- Воркер формально есть, но не обрабатывает задачи
- Нет ни одного теста или тесты падают
- README отсутствует или содержит только «запустите docker-compose»
- Прямые SQL-запросы вместо ORM без объяснения, зачем это сделано

Вопросы?

Если что-то в условии неясно — пишите, уточним. Намеренно неоднозначных требований нет.

Удачи!